Lab 04 · Despliegue canary y rollback automático con CodeDeploy

Duración estimada: 60 minutos **Sesión:** 2 **Entorno:** GitHub Codespaces con un emulador local de AWS · sin cuenta de AWS

Objetivo

Desplegar versiones nuevas de una función Lambda sin cortar el servicio y con red de seguridad. Primero moverás tráfico a mano entre dos versiones para entender qué es un canary; después dejarás que CodeDeploy lo haga por ti, con unas **pruebas de humo automáticas** que se ejecutan antes de dar tráfico a la versión nueva. Una versión sana llegará a producción; una versión defectuosa será detenida y el servicio seguirá en la versión anterior sin que nadie intervenga.

Cómo funciona el entorno

Trabajas en el mismo Codespace que en los labs anteriores. El emulador ejecuta las funciones Lambda de verdad, cada una en su contenedor, con versiones y alias igual que en AWS, y CodeDeploy mueve el tráfico del alias entre versiones e invoca la función de pruebas de humo (el *hook*) antes de hacerlo.

> **Diferencias con AWS real en esta práctica.** En el emulador, la espera del canary (los 5 minutos de `LambdaCanary10Percent5Minutes`) se comprime a unos segundos, y CodeDeploy **no** vigila alarmas de CloudWatch, porque el emulador no genera las métricas de errores de Lambda. Por eso en esta práctica la red de seguridad es el hook de pruebas de humo, que sí funciona igual que en AWS. Al final tienes cómo se haría con SAM, alarmas y ventana de observación en una cuenta real, que es la forma habitual en producción.

Requisitos

- El Codespace del curso abierto y el emulador respondiendo: `aws sts get-caller-identity`.

Punto de partida

La práctica es independiente. En el paso 1 creas los roles de servicio.

Paso 1 · Roles de servicio

Necesitas tres identidades: la de la función, la de CodeDeploy y la del hook de pruebas, que debe poder invocar la función y comunicar a CodeDeploy el resultado.

```
        mkdir -p ~/lab04 && cd ~/lab04

        confianza() { echo "{\"Version\":\"2012-10-17\",\"Statement\":[{\"Effect\":\"Allow\",\"Principal\":
{\"Service\":\"$1\"},\"Action\":\"sts:AssumeRole\"}]}"; }

        aws iam create-role --role-name lab04-funcion --assume-role-policy-document "$(confianza lambda.amazonaws.co
query Role.Arn --output text
        aws iam attach-role-policy --role-name lab04-funcion \
          --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

        aws iam create-role --role-name lab04-codedeploy --assume-role-policy-document "$(confianza
codedeploy.amazonaws.com)" --query Role.Arn --output text
        aws iam attach-role-policy --role-name lab04-codedeploy \
          --policy-arn arn:aws:iam::aws:policy/service-role/AWSCodeDeployRoleForLambda

        aws iam create-role --role-name lab04-hook --assume-role-policy-document "$(confianza lambda.amazonaws.com)"
query Role.Arn --output text
        aws iam attach-role-policy --role-name lab04-hook \
          --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
        aws iam put-role-policy --role-name lab04-hook --policy-name pruebas-humo --policy-document '{
          "Version": "2012-10-17",
          "Statement": [
```

```
            { "Effect": "Allow", "Action": "lambda:InvokeFunction", "Resource": "arn:aws:lambda:*:*:function:lab04-
saludo*" },
            { "Effect": "Allow", "Action": ["codedeploy:PutLifecycleEventHookExecutionStatus",
"codedeploy:GetDeployment"], "Resource": "*" }
          ]}'

        export ROL_FN=$(aws iam get-role --role-name lab04-funcion --query Role.Arn --output text)
        export ROL_CD=$(aws iam get-role --role-name lab04-codedeploy --query Role.Arn --output text)
        export ROL_HOOK=$(aws iam get-role --role-name lab04-hook --query Role.Arn --output text)
```

Paso 2 · La función y su versión 1

La función devuelve su versión. Tiene además una «tasa de error» que usarás más adelante para simular un despliegue defectuoso.

```
        mkdir -p fn
        cat > fn/index.py <<'EOF2'
        import json
        import random

        VERSION = "1.0.0"
        TASA_ERROR = 0.0


        def handler(event, context):
            if random.random() < TASA_ERROR:
                raise RuntimeError(f"Fallo simulado en la versión {VERSION}")
            return {"statusCode": 200, "body": json.dumps({"mensaje": "Hola", "version": VERSION})}
        EOF2

        (cd fn && rm -f ../fn.zip && python -m zipfile -c ../fn.zip index.py)
        aws lambda create-function --function-name lab04-saludo --runtime python3.12 \
          --handler index.handler --role $ROL_FN --zip-file fileb://fn.zip --query State --output text
        aws lambda wait function-active-v2 --function-name lab04-saludo

        aws lambda publish-version --function-name lab04-saludo --description "1.0.0" --query Version --output text
        aws lambda create-alias --function-name lab04-saludo --name live --function-version 1 --query AliasArn --out
text
```

> **Versiones y alias.** Una **versión** es una foto inmutable del código y la configuración: la versión 1 será siempre la 1.0.0. Un **alias** es un nombre estable (`live`) que apunta a una versión, y es lo que usan los clientes. Desplegar no es cambiar el código de la función: es publicar una versión nueva y mover el alias. Y un alias puede repartir el tráfico entre **dos** versiones, que es la base de un canary.

Define dos funciones de ayuda: `publicar` sube el código y publica una versión nueva, y `trafico` invoca el alias `live` las veces que le digas y resume qué versión respondió y cuántas veces falló.

```
        publicar() {   # uso: publicar <versión> <tasa_error>
          sed -i "s/^VERSION = .*/VERSION = \"$1\"/; s/^TASA_ERROR = .*/TASA_ERROR = $2/" fn/index.py
          (cd fn && rm -f ../fn.zip && python -m zipfile -c ../fn.zip index.py)
          aws lambda update-function-code --function-name lab04-saludo --zip-file fileb://fn.zip --query LastUpdateS
--output text
          aws lambda wait function-updated-v2 --function-name lab04-saludo
          aws lambda publish-version --function-name lab04-saludo --description "$1" --query Version --output text
        }

        trafico() {   # uso: trafico <número de peticiones>
          for i in $(seq 1 $1); do
            aws lambda invoke --function-name lab04-saludo:live --payload '{}' \
              --cli-binary-format raw-in-base64-out /tmp/respuesta.json \
              --query "[ExecutedVersion,FunctionError]" --output text
          done | sort | uniq -c | awk '{printf "  %3d peticiones -> versión %s %s\n", $1, $2, ($3=="None"?"OK":"ERRO
        }

        trafico 10
```

> **Punto de control 1.** Las 10 peticiones las responde la versión 1 sin errores.

Paso 3 · Un canary a mano

Publica la versión 1.1.0 (sana) y manda a ella solo el 20 % del tráfico. Esto es exactamente lo que hace un canary: la versión nueva recibe una parte pequeña del tráfico real mientras la antigua sigue sirviendo al resto.

```
publicar 1.1.0 0.0

aws lambda update-alias --function-name lab04-saludo --name live --function-version 1 \
  --routing-config AdditionalVersionWeights={2=0.2} \
  --query "[FunctionVersion,RoutingConfig.AdditionalVersionWeights]" --output json

trafico 30
```

> **Punto de control 2.** Aproximadamente una de cada cinco peticiones la responde la versión 2. Si fuera defectuosa, solo afectaría a ese 20 %.

Hacerlo a mano tiene dos problemas: alguien tiene que vigilar y alguien tiene que acordarse de deshacerlo. Devuelve todo el tráfico a la versión 1 y deja que, a partir de ahora, lo haga CodeDeploy.

> **Diferencia con AWS real.** En AWS bastaría con `aws lambda update-alias ... --routing-config '{"AdditionalVersionWeights": {}}'`, pero el emulador no borra así el reparto, de modo que aquí recreamos el alias apuntando solo a la versión 1.

```
aws lambda delete-alias --function-name lab04-saludo --name live
aws lambda create-alias --function-name lab04-saludo --name live --function-version 1 \
  --query "[FunctionVersion,RoutingConfig]" --output json
trafico 10
```

Paso 4 · Las pruebas de humo (hook `BeforeAllowTraffic`)

CodeDeploy puede invocar una función tuya **antes** de dar tráfico a la versión nueva. Esa función prueba la versión nueva directamente, sin pasar por el alias, y le dice a CodeDeploy si puede continuar (`Succeeded`) o si debe abortar (`Failed`). Si aborta, el alias ni se toca.

```
mkdir -p hook
cat > hook/index.py <<'EOF2'
import json

import boto3

lam = boto3.client("lambda")
codedeploy = boto3.client("codedeploy")

PETICIONES = 10


def version_destino(deployment_id):
    """Lee del despliegue qué versión se va a publicar; si no puede, prueba $LATEST."""
    try:
        d = codedeploy.get_deployment(deploymentId=deployment_id)["deploymentInfo"]
        appspec = json.loads(d["revision"]["appSpecContent"]["content"])
        recurso = list(appspec["Resources"][0].values())[0]
        return str(recurso["Properties"]["TargetVersion"])
    except Exception:
        return "$LATEST"


def handler(event, context):
```

```
            version = version_destino(event["DeploymentId"])
            errores = 0
            for _ in range(PETICIONES):
                r = lam.invoke(FunctionName="lab04-saludo", Qualifier=version, Payload=b"{}")
                if r.get("FunctionError"):
                    errores += 1
            estado = "Succeeded" if errores == 0 else "Failed"
            print(f"Pruebas de humo sobre la versión {version}: {errores}/{PETICIONES} errores -> {estado}")
            codedeploy.put_lifecycle_event_hook_execution_status(
                deploymentId=event["DeploymentId"],
                lifecycleEventHookExecutionId=event["LifecycleEventHookExecutionId"],
                status=estado,
            )
            return {"version": version, "errores": errores, "estado": estado}
        EOF2

        (cd hook && rm -f ../hook.zip && python -m zipfile -c ../hook.zip index.py)
        aws lambda create-function --function-name lab04-pruebas-humo --runtime python3.12 \
          --handler index.handler --role $ROL_HOOK --zip-file fileb://hook.zip --timeout 60 --query State --output t
        aws lambda wait function-active-v2 --function-name lab04-pruebas-humo
```

> **Ojo con el criterio.** Este hook exige **cero** errores en 10 peticiones. Es un criterio estricto a propósito: una prueba de humo no mide la calidad, solo detecta que algo está claramente roto antes de exponerlo a usuarios.

Paso 5 · Aplicación y grupo de despliegue de CodeDeploy

```
        aws deploy create-application --application-name lab04 --compute-platform Lambda

        aws deploy create-deployment-group --application-name lab04 --deployment-group-name live \
          --service-role-arn $ROL_CD \
          --deployment-config-name CodeDeployDefault.LambdaCanary10Percent5Minutes \
          --deployment-style deploymentType=BLUE_GREEN,deploymentOption=WITH_TRAFFIC_CONTROL \
          --auto-rollback-configuration enabled=true,events=DEPLOYMENT_FAILURE \
          --query deploymentGroupId --output text
```

Opción	Valor	Para qué sirve
`compute-platform`	`Lambda`	CodeDeploy no copia ficheros: mueve el tráfico de un alias.
`deployment-config-name`	`LambdaCanary10Percent5Minutes`	10 % del tráfico a la versión nueva, espera 5 minutos y, si todo va bien, el 100 %.
`auto-rollback-configuration`	`DEPLOYMENT_FAILURE`	Si el despliegue falla, el tráfico vuelve a la versión anterior.

Cada despliegue se describe con un **AppSpec**: qué alias, desde qué versión, hasta qué versión y qué hooks ejecutar. La función `desplegar` lo genera, lanza el despliegue y te enseña segundo a segundo cómo se reparte el alias:

```
        desplegar() {   # uso: desplegar <versión destino>
          ACTUAL=$(aws lambda get-alias --function-name lab04-saludo --name live --query FunctionVersion --output te
          cat > appspec.json <<EOF3
        {"version": 0.0,
         "Resources": [{"saludo": {"Type": "AWS::Lambda::Function",
           "Properties": {"Name": "lab04-saludo", "Alias": "live", "CurrentVersion": "$ACTUAL", "TargetVersion": "$1
         "Hooks": [{"BeforeAllowTraffic": "lab04-pruebas-humo"}]}
        EOF3
          python -c "import json; print(json.dumps({'revisionType': 'AppSpecContent', 'appSpecContent': {'content':
open('appspec.json').read()}}))" > revision.json
          DEP=$(aws deploy create-deployment --application-name lab04 --deployment-group-name live \
            --revision file://revision.json --query deploymentId --output text)
          echo "Despliegue $DEP: versión $ACTUAL -> versión $1"
          for i in $(seq 1 60); do
            S=$(aws deploy get-deployment --deployment-id $DEP --query deploymentInfo.status --output text)
            A=$(aws lambda get-alias --function-name lab04-saludo --name live \
              --query "[FunctionVersion,RoutingConfig.AdditionalVersionWeights]" --output json | tr -d ' \n')
            echo "  $(date +%T)  $S  alias=$A"
            case "$S" in Succeeded|Failed|Stopped) break;; esac
            sleep 1
```

```
  done
  aws deploy get-deployment --deployment-id $DEP \
    --query "deploymentInfo.[status,errorInformation.code,errorInformation.message]" --output text
}
```

Paso 6 · Desplegar una versión sana

La versión 2 (1.1.0) ya está publicada desde el paso 3. Despliégala con CodeDeploy:

```
desplegar 2
trafico 10
```

Mira lo que ha pasado en la salida: antes de la primera línea, CodeDeploy ya ha ejecutado las pruebas de humo contra la versión 2; como las ha superado, el alias pasa a repartir el tráfico, `["1",{"2":0.1}]` (el canary: 10 % a la versión nueva) y, al terminar la espera, `["2",null]`: todo el tráfico en la versión 2.

> **Punto de control 3.** El despliegue termina en `Succeeded` y las 10 peticiones las responde la versión 2. En AWS real, entre el 10 % y el 100 % pasarían 5 minutos; el emulador lo comprime a unos segundos.

Consulta el log de las pruebas de humo:

```
aws logs filter-log-events --log-group-name /aws/lambda/lab04-pruebas-humo \
  --query "events[].message" --output text | tr '\t' '\n' | grep "Pruebas de humo"
```

Paso 7 · Desplegar una versión defectuosa

Ahora alguien publica la 2.0.0, que falla en la mitad de las peticiones:

```
publicar 2.0.0 0.5
desplegar 3
```

> **Punto de control 4.** El despliegue termina en `Failed` con el código `HOOK_EXECUTION_FAILURE` y el alias **no se ha movido** de la versión 2: las pruebas de humo detectaron los errores antes de que un solo usuario recibiera tráfico de la versión 3.

Compruébalo desde el punto de vista de los clientes y en el log del hook:

```
trafico 20
aws logs filter-log-events --log-group-name /aws/lambda/lab04-pruebas-humo \
  --query "events[].message" --output text | tr '\t' '\n' | grep "Pruebas de humo"
aws deploy list-deployments --application-name lab04 --deployment-group-name live \
  --query deployments --output text | tr '\t' '\n' | while read d; do
    aws deploy get-deployment --deployment-id $d \
      --query "deploymentInfo.[deploymentId,status,errorInformation.code]" --output text
  done
```

> **Punto de control final.** Las 20 peticiones las responde la versión 2 sin errores, el log del hook muestra una prueba superada y otra fallida, y el historial de CodeDeploy tiene un despliegue `Succeeded` y otro `Failed`. El servicio no se ha enterado del despliegue defectuoso.

Arregla la versión y despliégala para cerrar el ciclo:

```
publicar 2.0.1 0.0
desplegar 4
trafico 10
```

Paso 8 · Cómo sería en AWS real

En una cuenta real, todo lo que has hecho a mano con la CLI se declara en la plantilla SAM de la función, y la red de seguridad tiene **dos** capas: el hook de pruebas de humo antes de dar tráfico, y una **alarma** de CloudWatch que CodeDeploy vigila durante los 5 minutos del canary, mientras el 10 % de los usuarios reales usa la versión nueva. Si la alarma salta, CodeDeploy devuelve todo el tráfico a la versión anterior y CloudFormation marca la actualización del stack como fallida.

```
        Resources:
          FuncionSaludo:
            Type: AWS::Serverless::Function
            Properties:
              Runtime: python3.12
              Handler: index.handler
              CodeUri: fn/
              AutoPublishAlias: live                      # publica versión y mueve el alias en cada cambio
              DeploymentPreference:
                Type: Canary10Percent5Minutes
                Alarms:
                  - !Ref AlarmaErrores                      # si salta durante el canary: rollback
                Hooks:
                  PreTraffic: !Ref FuncionPruebasHumo       # el hook de este laboratorio

          AlarmaErrores:
            Type: AWS::CloudWatch::Alarm
            Properties:
              Namespace: AWS/Lambda
              MetricName: Errors
              Dimensions:
                - { Name: FunctionName, Value: !Ref FuncionSaludo }
                - { Name: Resource, Value: !Sub "${FuncionSaludo}:live" }
              Statistic: Sum
              Period: 60
              EvaluationPeriods: 1
              Threshold: 0
              ComparisonOperator: GreaterThanThreshold
              TreatMissingData: notBreaching
```

Para que la alarma sirva de algo, la versión nueva tiene que recibir tráfico durante la ventana del canary. En producción, si el tráfico es bajo (por la noche, por ejemplo), se complementa con pruebas sintéticas como CloudWatch Synthetics.

Reto opcional

Cambia la configuración del grupo de despliegue a `CodeDeployDefault.LambdaLinear10PercentEvery1Minute` con `aws deploy update-deployment-group` y despliega una versión nueva sana. ¿Qué diferencia ves en cómo cambia el alias respecto al canary?

Preguntas de reflexión

1. ¿Qué protege el hook de pruebas de humo y qué protege la alarma? ¿Por qué en producción se usan los dos?
2. Si tu función tiene muy poco tráfico, ¿qué riesgo tiene confiar solo en la alarma durante un canary de 5 minutos?
3. Rollback de código frente a rollback de datos: si la versión defectuosa hubiera cambiado el formato de los registros de una tabla, ¿bastaría con devolver el alias a la versión anterior?

Limpieza

```
        cd ~/lab04
        aws deploy delete-deployment-group --application-name lab04 --deployment-group-name live
        aws deploy delete-application --application-name lab04
        aws lambda delete-function --function-name lab04-saludo
        aws lambda delete-function --function-name lab04-pruebas-humo
        for r in lab04-funcion lab04-codedeploy lab04-hook; do
          for p in $(aws iam list-attached-role-policies --role-name $r --query "AttachedPolicies[].PolicyArn" --out
text); do
            aws iam detach-role-policy --role-name $r --policy-arn $p
```

```
    done
    for p in $(aws iam list-role-policies --role-name $r --query PolicyNames --output text); do
      aws iam delete-role-policy --role-name $r --policy-name $p
    done
    aws iam delete-role --role-name $r
  done
```

Detén el Codespace al terminar: **Code → Codespaces → ⋯ → Stop codespace**.